Alexandria University • Faculty of Engineering
Computer and Communications Engineering Program
Spring 2026 | CSE 476: Data Mining

Final Project: Building a Recommendation System

Assigned: May 7, 2026 | Due: May 20 , 2026

Dr. Nada Salah | Eng. Nabeel Zamel | Eng. Aya Ragaa

Overview

In this final project you will design and implement a complete recommendation system pipeline from scratch. Recommendation systems are one of the most impactful applications of data mining, powering platforms such as Netflix, Spotify, Amazon, and YouTube. You will apply the three main approaches — Content-Based Filtering, Collaborative Filtering (User-User and Item-Item), and Latent-Factor Matrix Factorization — and rigorously compare their strengths and weaknesses on a real-world dataset.

Dataset

You will use the MovieLens dataset for this project. Download the ml-latest-small version (100,000 ratings, 9,000 movies, 600 users) from:

https://grouplens.org/datasets/movielens/latest/

The dataset contains the following files:

- ratings.csv — userId, movieId, rating (0.5 to 5.0), timestamp
- movies.csv — movieId, title, genres
- tags.csv — userId, movieId, tag, timestamp
- links.csv — movieId, imdbId, tmdbId (optional, for enrichment)

Part 1 — Dataset Loading & Exploratory Data Analysis

Part 1: EDA

1.1 Loading & Preprocessing

- Load all relevant CSV files and merge them appropriately.
- Report the number of users, movies, and ratings.
- Handle any missing values or duplicate entries.

- Construct the User-Item utility matrix (users × movies) with ratings as values and NaN for unrated pairs.
- Report the sparsity of the utility matrix (% of missing entries).

1.2 Exploratory Analysis

- Plot the distribution of ratings (histogram). Comment on any skew.
- Plot the distribution of ratings per user and ratings per movie. Identify power-law behaviour.
- Visualize the genre distribution across all movies.
- Identify and discuss the cold-start problem: how many users have fewer than 5 ratings? How many movies?
- Apply mean-centering to the utility matrix (subtract each user's mean rating from their ratings) and show how this changes the distribution.

Part 2 — Content-Based Filtering

Part 2: Content-Based

2.1 Item Profile Construction

Build an item profile for each movie based on its genre information. Each movie should be represented as a binary (or weighted) feature vector over all possible genres.

- Extract all unique genres and build a genre vector per movie.
- Additionally, incorporate the movie's average rating and number of ratings as additional features.
- (Optional enrichment) If you choose to use the tags.csv file, apply TF-IDF to the tags to generate a richer item representation and concatenate it with the genre features.

Note: *You may use sklearn's TfidfVectorizer only for TF-IDF computation; the item profile assembly must be done manually.*

2.2 User Profile Construction

Build a user profile for each user as a weighted average of the item profiles of the movies they have rated, weighted by their ratings (shifted so that a rating of 3 = neutral).

- Show an example user profile vector for at least two users.
- Visualize the user profiles using dimensionality reduction (PCA to 2D) and plot them — do similar users cluster together?

2.3 Recommendation & Evaluation

- For each user, compute the cosine similarity between their user profile and all unrated item profiles.

- Recommend the top-10 movies for at least 3 example users. Discuss whether the recommendations look reasonable.
- Evaluate using leave-one-out cross-validation: hide each user's last-rated movie and check if it appears in the top-10 recommendations. Report Precision@10 and Recall@10.

Part 3 — Collaborative Filtering

Part 3: Collaborative Filtering

3.1 Mean-Centering & Similarity

- Apply mean-centering to the utility matrix (per user). Explain why this is preferable over raw cosine similarity.
- Implement the overlapping-item cosine similarity (only consider items co-rated by both users/items when computing similarity).
- Justify your choice of similarity metric with a concrete numerical example from the data (as done in the lecture).

3.2 User-User Collaborative Filtering

Implement User-User CF from scratch (no external CF libraries).

- For a target user and unrated movie, find the N most similar users who have rated that movie and estimate the rating as a similarity-weighted average.
- Experiment with $N \in \{5, 10, 20, 50\}$ and report the effect on RMSE (on a held-out validation set — 20% of ratings).
- Generate top-10 movie recommendations for at least 3 users and discuss the results.
- Discuss: what happens for users with very few ratings (cold-start)?

3.3 Item-Item Collaborative Filtering

Implement Item-Item CF from scratch.

- Compute item-item similarity using the mean-centered utility matrix.
- For a target user and unrated movie, estimate the rating using the similarity-weighted average of the user's ratings of similar movies.
- Experiment with $N \in \{5, 10, 20, 50\}$ and report RMSE on the validation set.
- Compare Item-Item vs. User-User CF: which performs better on this dataset and why? (Hint: think about how many users vs. items there are and how stable each similarity is.)

Part 4 — Latent-Factor Model: Matrix Factorization

Part 4: Latent-Factor / MF

4.1 Implementation

Implement Matrix Factorization (MF) using Stochastic Gradient Descent (SGD) from scratch. The model should learn user latent vectors X (N × k) and item latent vectors Y (k × M) by minimizing the regularized loss:

$$L = (1/N) * \Sigma (r_{ui} - x_u^T y_i)^2 + \lambda (||x_u||^2 + ||y_i||^2)$$

- Initialize X and Y with small random values.
- Implement the SGD update rules for x_u and y_i (gradients are given in the lecture slides).
- Train for a configurable number of epochs and plot the training loss and validation RMSE per epoch.
- Experiment with latent dimensions $k \in \{10, 20, 50, 100\}$ and regularization $\lambda \in \{0.001, 0.01, 0.1\}$.
- Report the best configuration and its validation RMSE.

4.2 Analysis

- Visualize the learned item latent vectors using PCA (reduce to 2D). Color the points by genre. Do movies of the same genre cluster together?
- Find the 5 most similar movies to a movie of your choice using the cosine similarity of their latent vectors y_i. Do the results make intuitive sense?
- Discuss: how does the MF model handle new users or new items (the cold-start problem)? What strategies exist?
- Generate top-10 recommendations for 3 users and compare them to the recommendations from Parts 2 and 3.

Part 5 — Evaluation & Comparison

Part 5: Evaluation

Perform a systematic comparison of all three approaches on the same train/validation/test split (80% / 10% / 10%). The test split must remain unseen until final evaluation.

5.1 Rating Prediction Metrics

- Report RMSE and MAE on the test set for: Content-Based (predicted ratings from profile similarity scores), User-User CF (best N), Item-Item CF (best N), and Matrix Factorization (best k, λ).
- Plot a grouped bar chart comparing RMSE and MAE across all methods.

5.2 Ranking Metrics

- For each method, generate a ranked list of top-10 recommendations per user.
- Compute and report Precision@10 and Recall@10 for each method (use the test set ratings as ground truth, treating ratings ≥ 4.0 as relevant).
- Discuss the trade-off between RMSE-based and ranking-based evaluation.

Part 6 — Report

Part 6: Report

Submit a PDF report (4–8 pages, excluding figures and references). The report should be self-contained and readable without referring to the code. It must include:

- Introduction: the recommendation problem, the dataset, and your overall approach.
- Methodology: a clear description of each algorithm implemented, including the key equations.
- Results: all plots and tables from Parts 1–5, with captions and discussion.
- Conclusion: a summary of findings, limitations of each approach, and suggestions for improvement.
- References: cite any external resources, papers, or datasets used.

Bonus — Hybrid Recommendation System

Bonus: Hybrid System

Implement a hybrid recommender that combines at least two of the above approaches to improve performance. For example, you may combine content-based filtering with matrix factorization by initializing item latent vectors from genre features, or blend the predicted ratings of multiple models.

- Describe your hybridization strategy and justify the design choice.
- Report RMSE, MAE, Precision@10, and Recall@10 for the hybrid model on the test set.
- Compare to the individual models. Does the hybrid outperform all individual approaches?
- Discuss when hybrid systems are most beneficial and what the added complexity costs.

Submission Guidelines

- Teams of up to 3 students are allowed.
- Submit one Jupyter Notebook (.ipynb) containing all code, outputs, and inline comments.
- Submit one PDF report (Part 6). Both files must be submitted together.

- Teams may lose credit in the discussion session if any member cannot explain the work.
- Any cases of plagiarism or sharing of code between teams will result in a score of 0% for all involved teams and will be reported to the faculty.